Ciclos em Funções

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027

Baseado em slides de Luís Caires

Outline

1. Da Aula 5 às Funções Iterativas

2. Decompor um Cálculo Iterativo

3. Síntese

Da Aula 5 para hoje

Aula 5

- ciclos `while`;
- estado, condição e atualização;
- contadores e acumuladores;
- terminação e invariantes.

Nesta aula

- colocar ciclos dentro de funções;
- transformar valores concretos em parâmetros;
- dar contratos a cálculos iterativos;
- combinar funções num programa maior.

Um Cálculo Preso a um Único Caso

```
1 total = 0
2 atual = 1
3 while atual <= 10:
4     total = total + atual
5     atual = atual + 1
6 print(total)
```

- O ciclo calcula apenas a soma de `1` até `10`.
- O resultado é imediatamente apresentado e não pode ser composto noutro cálculo.
- Para mudar o limite, é necessário editar o próprio programa.

Encapsular um Cálculo Iterativo

```
1 def soma_ate(n: int) -> int:
2     total = 0
3     atual = 1
4     while atual <= n:
5         total = total + atual
6         atual = atual + 1
7     return total
```

- O valor concreto `10` tornou-se o parâmetro `n`.
- O ciclo é agora um cálculo com nome e resultado.
- Podemos chamar `soma_ate` com vários limites e usar o resultado noutras expressões.

Anatomia de uma Função Iterativa

```
1 def soma_ate(n: int) -> int:    # entrada
2     total = 0                  # acumulador
3     atual = 1                  # progresso
4     while atual <= n:          # condição
5         total = total + atual  # atualizar resultado
6         atual = atual + 1      # garantir progresso
7     return total               # resultado completo
```

Ligação à Aula 5

O ciclo mantém a mesma estrutura; a função acrescenta parâmetros, estado local, contrato e resultado reutilizável.

Parâmetros: O Que Deve Variar?

Queremos transformar este cálculo numa função reutilizável:

```
1 total = 0
2 valor = 3
3 while valor <= 30:
4     total = total + valor
5     valor = valor + 3
6 print(total)
```

- Que valores estão presos ao exemplo?
- Que parâmetros permitem somar múltiplos diferentes até limites diferentes?
- Que valor deve a função devolver?

Uma Solução Possível: Parâmetros do Ciclo

Problema: somar os múltiplos positivos de `passo` que não excedem `limite`.

```
1 def soma_multiplos(passo: int, limite: int) -> int:
2     total = 0
3     valor = passo
4     while valor <= limite:
5         total = total + valor
6         valor = valor + passo
7     return total
```

`soma_multiplos(3, 30)` reproduz o cálculo apresentado; outros argumentos reutilizam o mesmo ciclo.

Contrato Antes do Ciclo

Problema

Definir `contar_divisores(n)`, que recebe um inteiro positivo e devolve o número de divisores positivos de `n`.

Entrada: um inteiro `n > 0`.

Resultado: quantidade de inteiros entre `1` e `n` que dividem `n` exatamente.

Exemplos: `contar_divisores(1)== 1` e `contar_divisores(6)== 4`.

Questão

Que informação tem de mudar durante a execução?

Que Estado Devemos Manter?

Nome	Papel	Valor inicial
`d`	candidato a divisor	`1`
`contador`	divisores encontrados	`0`
`n`	limite e número testado	parâmetro

- O ciclo progride aumentando `d`.
- O contador só muda quando `n % d == 0`.
- Quando `d > n`, todos os candidatos foram examinados.

Implementar `contar_divisores`

```
1 def contar_divisores(n: int) -> int:
2     d = 1
3     contador = 0
4     while d <= n:
5         if n % d == 0:
6             contador = contador + 1
7         d = d + 1
8     return contador
```

- A condição do `if` decide se há uma contribuição.
- A atualização de `d` acontece em todas as iterações.

Trace de `contar_divisores(6)`

`d`	`6 % d == 0`	`contador` depois
1	`True`	1
2	`True`	2
3	`True`	3
4	`False`	3
5	`False`	3
6	`True`	4

À saída

`d == 7`; o ciclo termina e a função devolve `4`.

Análise: Onde Deve Ficar o `return`?

Compare as duas posições assinaladas:

```
1 def contar_divisores(n: int) -> int:
2     d = 1
3     contador = 0
4     while d <= n:
5         if n % d == 0:
6             contador = contador + 1
7         d = d + 1
8         # posição A
9     # posição B
```

Em que posição devemos escrever `return contador`? O que aconteceria na outra?

Resultado Esperado: `return` Depois do Ciclo

Problema: devolver a contagem apenas quando todos os candidatos tiverem sido testados.

- Na posição A, a primeira iteração terminaria imediatamente a chamada.
- Na posição B, o ciclo examina todos os valores de `1` até `n`.
- Portanto, **`return`** `contador` deve ficar depois do ciclo.

Regra de leitura

Um **`return`** termina a função, mesmo quando aparece dentro de um ciclo.

Testar Casos Pequenos e Fronteiras

```
1 assert contar_divisores(1) == 1
2 assert contar_divisores(2) == 2
3 assert contar_divisores(6) == 4
4 assert contar_divisores(12) == 6
```

- `1`: menor entrada permitida pelo contrato.
- `2`: número primo, com exatamente dois divisores.
- `6` e `12`: números compostos com contagens diferentes.

Desenhar uma Função Iterativa

1. Escrever o contrato: entrada válida e resultado pretendido.
2. Escolher o estado: progresso e resultado parcial.
3. Definir os valores iniciais.
4. Escrever a condição de continuação.
5. Atualizar o resultado e garantir progresso.
6. Devolver o resultado depois do ciclo.

Verificação

Testar primeiro exemplos pequenos que seja possível calcular à mão.

Outline

1. Da Aula 5 às Funções Iterativas
2. Decompor um Cálculo Iterativo
3. Síntese

Um Programa Precisa de Cálculos com Significado

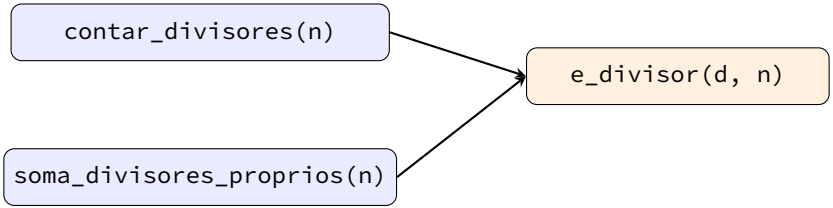
Para estudar os divisores de um número, podemos querer:

- contar quantos divisores existem;
- somar os divisores próprios;
- comparar resultados para diferentes números;
- reutilizar estes cálculos sem copiar os ciclos.

Motivação

`contar_divisores` e `soma_divisores_proprios` encapsulam ciclos completos. Cada nome representa um cálculo não trivial que pode ser testado e reutilizado.

Separar Responsabilidades



- As duas funções principais percorrem candidatos e acumulam resultados diferentes.
- Um pequeno auxiliar dá nome ao teste repetido dentro de ambos os ciclos.

Um Auxiliar Curto, mas Secundário

A Aula 4 já definiu o teste de divisibilidade:

```
1 def e_divisor(d: int, n: int) -> bool:
2     if d == 0:
3         return False
4     return n % d == 0
```

- Esta função não é a motivação principal para a decomposição.
- Serve para tornar legível uma condição usada dentro de funções iterativas maiores.
- O trabalho substancial continua em `contar_divisores` e `soma_divisores_proprios`.

Contrato da Soma dos Divisores Próprios

Nome: `soma_divisores_proprios`.

Entrada: um inteiro `n > 0`.

Resultado: soma dos divisores de `n` entre `1` e `n - 1`.

Estado do ciclo

`d` percorre os candidatos; `total` acumula os valores que satisfazem `e_divisor(d, n)`.

Fronteira

O próprio `n` não é testado, porque não é um divisor próprio.

Implementar `soma_divisores_proprios`

```
1 def soma_divisores_proprios(n: int) -> int:
2     d = 1
3     total = 0
4     while d < n:
5         if e_divisor(d, n):
6             total = total + d
7         d = d + 1
8     return total
```

- A estrutura do ciclo é semelhante à de `contar_divisores`.
- Mudam a fronteira e a informação acumulada.

Trace de `soma_divisores_proprios(6)`

`d`	É divisor?	`total` depois
1	sim	1
2	sim	3
3	sim	6
4	não	6
5	não	6

À saída

`d == 6`, `total == 6`; o próprio número não foi incluído.

O Mesmo Padrão, Resultados Diferentes

	`contar_divisores`	`soma_divisores_proprios`
progresso	`d = 1, ..., n`	`d = 1, ..., n-1`
estado	`contador`	`total`
contribuição	`+ 1`	`+ d`
resultado	quantidade	soma

Leitura

Reconhecer um padrão não significa esconder as diferenças importantes entre os contratos.

Previsão: Que Valores São Produzidos?

Sem executar, determine o resultado de cada chamada:

```
1 contar_divisores(1)
2 contar_divisores(6)
3 soma_divisores_proprios(1)
4 soma_divisores_proprios(6)
5 soma_divisores_proprios(8)
```

- Compare as fronteiras `d <= n` e `d < n`.
- A resposta fica para o slide seguinte.

Resultado Esperado: Contar e Somar

Problema: aplicar dois contratos iterativos diferentes ao mesmo domínio.

```
contar_divisores(1)              # 1
contar_divisores(6)              # 4
soma_divisores_proprios(1)     # 0
soma_divisores_proprios(6)     # 6
soma_divisores_proprios(8)     # 7
```

Testar Cada Cálculo Separadamente

```
1 assert contar_divisores(1) == 1
2 assert contar_divisores(2) == 2
3 assert contar_divisores(6) == 4
4
5 assert soma_divisores_proprios(1) == 0
6 assert soma_divisores_proprios(6) == 6
7 assert soma_divisores_proprios(8) == 7
```

Vantagem

Cada função pode ser verificada sem depender de interação com o utilizador nem de outro ciclo.

Uma Função, Um Cálculo Coerente

- `contar_divisores` controla um ciclo e produz uma contagem.
- `soma_divisores_proprios` controla outro ciclo e produz uma soma.
- `e_divisor` nomeia apenas uma condição partilhada pelos dois cálculos.

Critério prático

Uma função é especialmente útil quando encapsula um cálculo completo com estado próprio, fronteiras e resultado observável.

Quando Vale a Pena Criar uma Função?

- Quando um bloco realiza um cálculo com significado e contrato próprios.
- Quando o cálculo pode ser chamado com entradas diferentes.
- Quando queremos testá-lo sem executar o programa inteiro.
- Quando a implementação tem estado, decisões ou um ciclo que não deve ser repetido pelos clientes.

Sem funções artificiais

Nem toda a expressão precisa de ser transformada numa função. O objetivo é isolar responsabilidades reais, não multiplicar nomes sem benefício.

Outline

1. Da Aula 5 às Funções Iterativas
2. Decompor um Cálculo Iterativo
3. Síntese

Do Ciclo ao Programa Estruturado

1. Recuperar o padrão de ciclo adequado ao problema.
2. Encapsular o ciclo numa função com contrato explícito.
3. Dar nomes distintos ao progresso e ao resultado parcial.
4. Extrair apenas condições auxiliares que tornem o ciclo mais legível.
5. Testar cada cálculo iterativo de forma independente.

Ideia central

As funções não substituem os ciclos: dão-lhes um nome, um contrato e um lugar claro no programa.

Conclusão

Hoje vimos

- ciclos `while` encapsulados em funções
- contratos, estado local e resultados iterativos
- decomposição de cálculos substanciais e reutilizáveis

A seguir

- tuplos para representar dados compostos
- construtores e seletores sobre representações
- invariantes e barreiras de abstracção